ALOHAS

ANALYTICS ENGINEER CASE STUDY

Data Quality & Business Analysis

A comprehensive analysis of ALOHAS' data to ensure quality and extract actionable business insights.

Table of Contents

0. Data Quality Checks

All the data quality checks code is inside **analysis/data_quality.sql**

0.1 Duplicate products or shipments

I checked for duplicate records in both `dim_product` and `fct_shipment`, and no duplicates were found. This is important because both tables define a unique key (`sku` for products and `shipment_id` for shipments).

If duplicates were present, joining `fct_sale_order_line` with either table could multiply rows and consequently inflate metrics such as sales, quantities, returns, or shipping costs. This would be particularly problematic because `fct_sale_order_line` links to products through `sku` and to shipments through `shipment_id`.

0.2 SKUs in sales that don't exist in dim_product

I found **922 distinct SKUs** in `fct_sale_order_line`. Of these, **722 (78.3%) do not exist in `dim_product`**, meaning that only 200 SKUs (21.7%) can be matched to the product dimension.

The unmatched SKUs follow a clear pattern: they all start with `SKU-9`, while the SKUs present in `dim_product` start with `SKU-0`. This suggests that the issue is systematic rather than a small number of isolated missing products.

This creates a significant referential integrity issue. Since `sku` is the key linking `fct_sale_order_line` to `dim_product`, these unmatched sales cannot be reliably enriched with product attributes such as `name`, `category`, or `cost`.

Any analysis by product category or contribution margin using `dim_product.cost` would therefore be incomplete or potentially misleading unless these unmatched SKUs are handled explicitly.

0.3 Shipment_ids in sales that don't exist in fct_shipment

I found **24,741 distinct `shipment_id` values** in `fct_sale_order_line`. Of these, **499 (2.0%) do not exist in `fct_shipment`**.

The unmatched shipment IDs follow a clear pattern: they fall within the `SHP-90...` to `SHP-99...` range, while the shipment IDs that exist in `fct_shipment` start with `SHP-00...`. This suggests the missing references are systematic rather than isolated records.

This creates a referential integrity issue because `shipment_id` is the key linking `fct_sale_order_line` to `fct_shipment`. As a result, the affected sales cannot be

reliably enriched with shipment attributes such as `shipping_method`, `shipping_cost`, or destination country.

This is particularly relevant for the contribution margin analysis, as the shipping cost is one of the costs that needs to be incorporated into the calculation.

0.4 Check how many records and sales volume we have for each year-month

I checked the number of records and sales volume by year-month. The dataset covers **May 2024 to May 2026**, with no gaps in the monthly data.

The expected seasonal pattern is also visible, with clear **November and December peaks in both 2024 and 2025**. This is consistent with the seasonal pattern described in the case study, which highlights a Nov/Dec peak in sales.

0.5 Returns > sold

I checked for cases where `quantity_returned` is greater than `quantity_sold`, which would be logically inconsistent since a customer cannot return more units than were originally sold on the same order line.

No such cases were found. Therefore, there are no apparent inconsistencies between sold and returned quantities in the dataset, and no further investigation is required for this check.

0.6 Negative quantities

I checked both `quantity_sold` and `quantity_returned` for negative values. No negative values were found.

This confirms that the quantity fields contain only non-negative values, with no apparent data-quality issues related to negative sales or return quantities.

0.7 Gross sales validation

`gross_sale` is provided directly in the source data, but it can also be independently calculated as `dim_product.base_price × quantity_sold`.

I compared the source `gross_sale` against the calculated value and found **no discrepancies**. This confirms that the source gross sales values are consistent with the underlying product prices and quantities sold.

0.8 Net sales validation

`net_sales` is provided directly in the source data, but it can also be independently calculated as `gross_sale − taxes`.

I compared the source `net_sales` values against the calculated values and found **no discrepancies**. This confirms that the net sales values are consistent with the underlying gross sales and tax amounts.

1. Channel Sales

For this question, I created two queries in **business_questions/channel_sales.sql**.

1.1 Time Grain

Query **1A** measures channel performance at a **monthly grain**, with one row per channel and month.

I chose monthly as the primary time grain because daily data would be too noisy for a two-year business-performance view, while quarterly data would be too coarse to identify changes in channel performance. Monthly data provides a good balance for analysing **channel mix, seasonality, growth, and returns**. It is also particularly relevant given that the case study explicitly highlights a November/December sales peak.

1.2 Comparing channels on a like-for-like basis

I created a new metric, `realized_net_sales`, to account for the impact of returns:

```
realized_net_sales = net_sales – estimated value of returned units
```

I chose `net_sales` as the starting point rather than `gross_sale` because of the different tax treatment across channels. According to the source definition, Wholesale has zero taxes, while the other channels are subject to taxes. Using `gross_sale` would therefore make the comparison less meaningful, as it would include taxes for some channels but not others.

However, Query **1B**, which aggregates performance at the channel level, highlighted another important difference: **Online has a 17.9% return rate compared with only 4.2% for Wholesale**. Looking only at `net_sales` would therefore make Online appear stronger than it actually is after accounting for returned units.

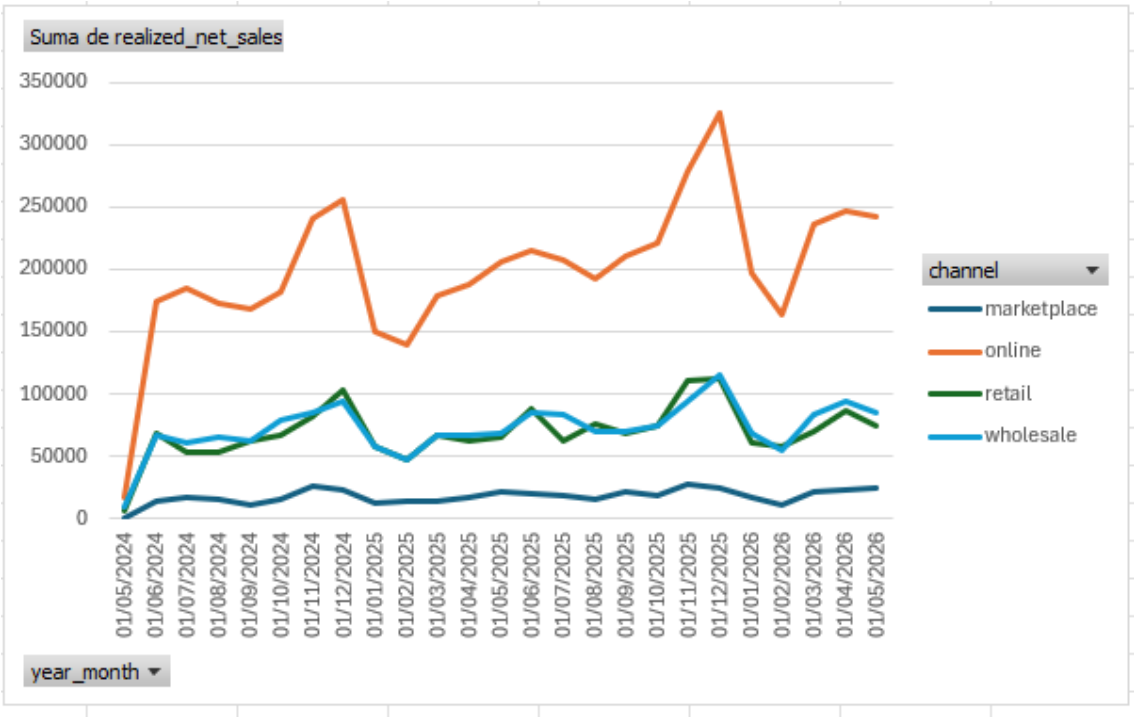
Returns also have an impact on the overall channel mix:

Channel	Net sales mix	Realized net sales mix
Online	57,92%	55,58%
Wholesale	18,02%	20,16%
Retail	19,14%	19,32%
Marketplace	4,92%	4,94%

After accounting for returns, Online's share decreases from **57.92% to 55.58%**, while Wholesale increases from **18.02% to 20.16%**. This is a small but meaningful redistribution of the channel mix and results in Wholesale surpassing Retail in realized net sales.

1.3 Is one channel quietly growing while another leaks?

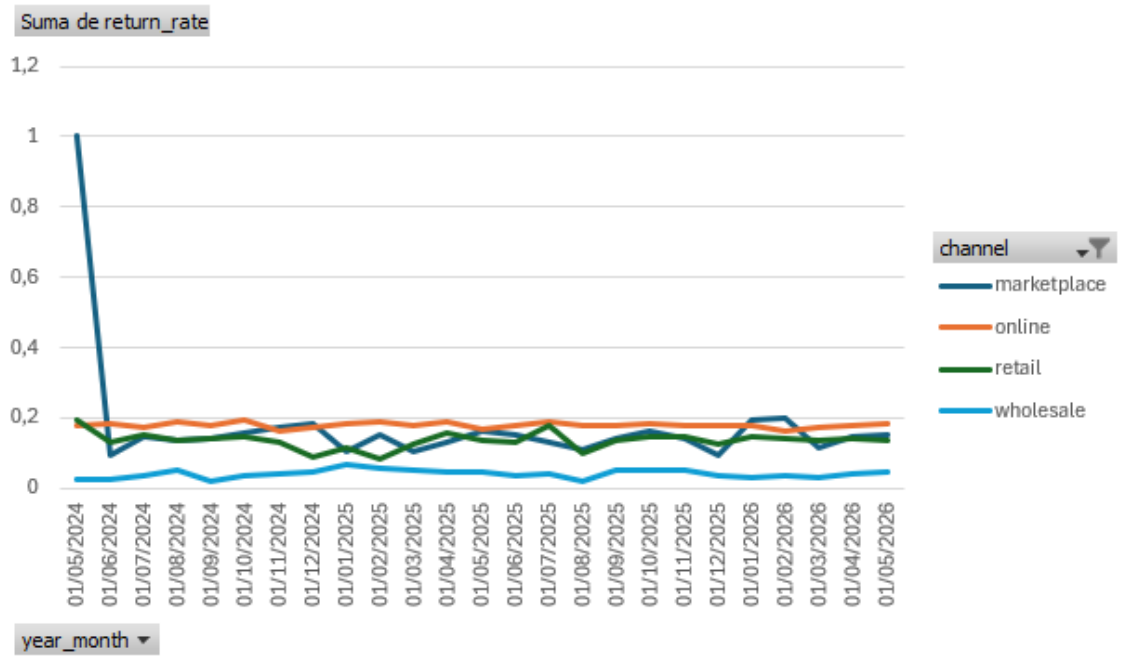
To investigate this, I created a monthly `realized_net_sales` chart by channel, available in **Dashboard.xlsx**.



The chart does not suggest that one channel is quietly growing while another is consistently declining. Instead, the four channels broadly follow the same overall pattern, with peaks and dips occurring around similar months.

The main difference is the **scale and volatility** of the channels. Online, being the largest channel, naturally shows the largest absolute fluctuations, while Marketplace, the smallest channel, shows smaller movements. However, there is no clear evidence from the overall trend of a channel structurally growing at the expense of another.

I also created a monthly **return-rate by channel** chart. This provides a useful second perspective because it can highlight unusual return behaviour that would be hidden when looking only at sales.



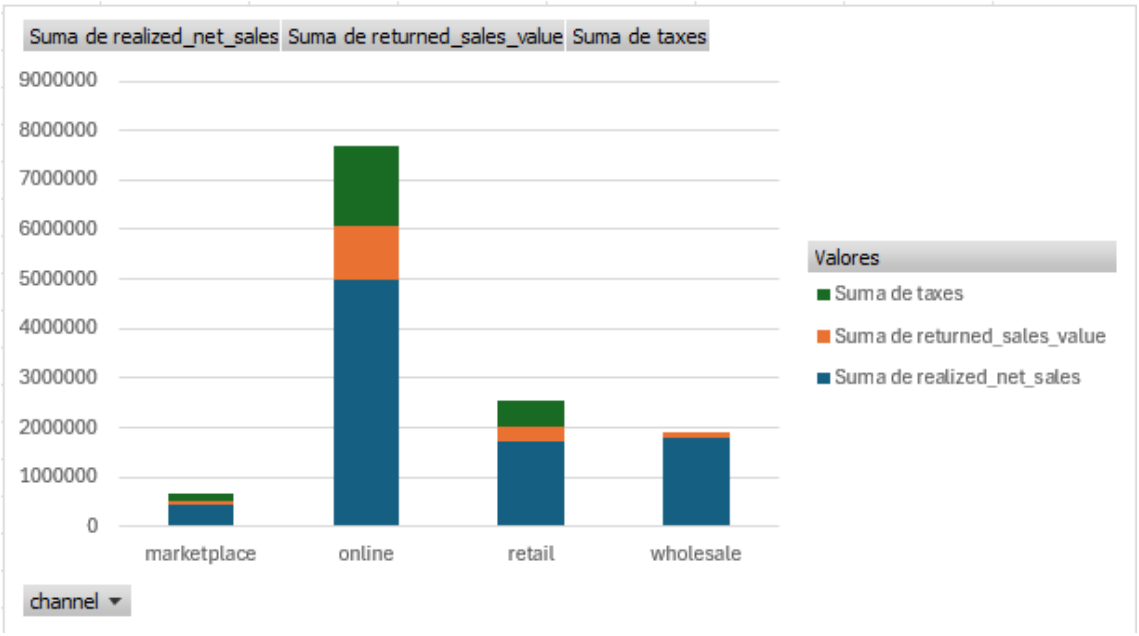
For example, Marketplace has a **100% return rate in May 2024**. However, only **3 units were sold and all 3 were returned**, so the percentage is highly sensitive to the very small volume. One possible explanation is that Marketplace had only one order at the end of the month and that order was subsequently returned. This is a good example of why return rates should always be interpreted alongside the underlying sales volume.

1.4 Sales waterfall / breakdown

Finally, I created a stacked bar chart showing how gross_sale is broken down into:

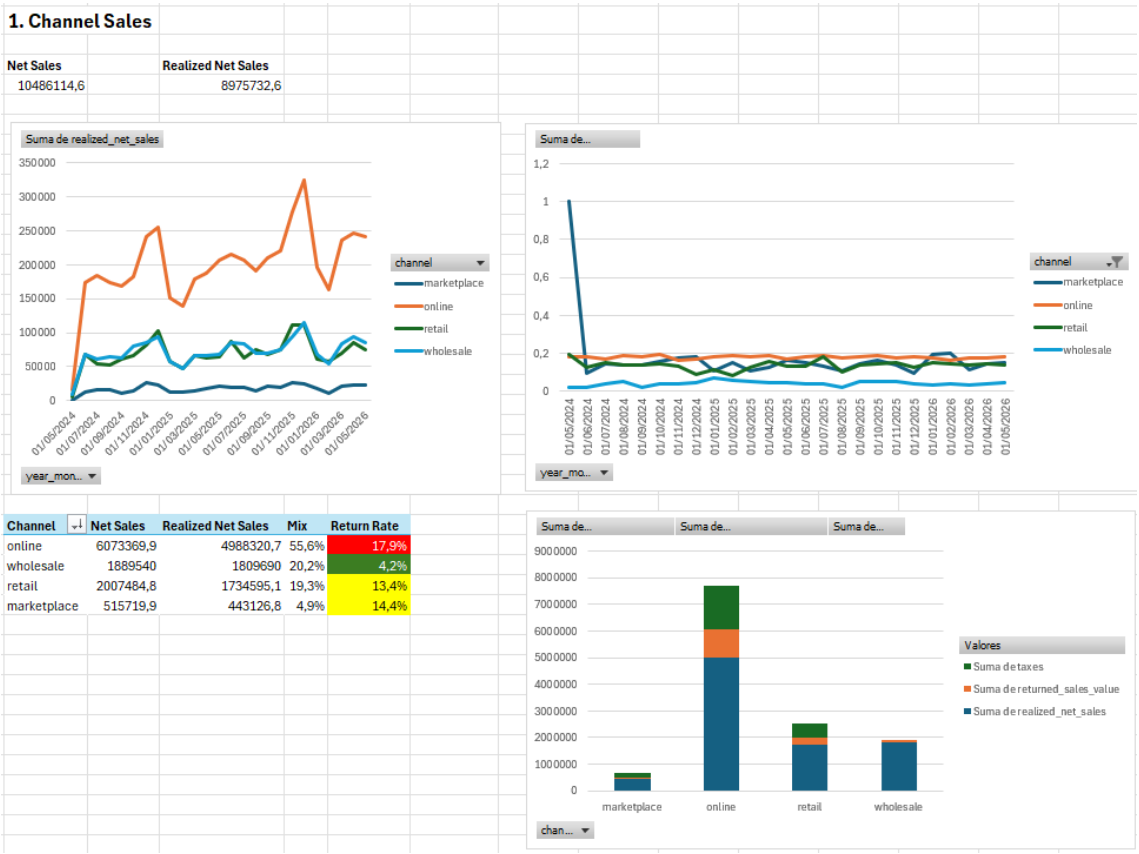
- Taxes
- Returned sales value
- Realized net sales

This provides a clear bridge from gross sales to the amount of sales actually retained after taxes and estimated returns. This chart helps distinguish between revenue generated initially and the portion that remains after accounting for taxes and returns.



1.5 Executive view

The final report for this business question is designed to answer the question differently depending on the audience.



1. Channel Sales

Net Sales	Realized Net Sales
10486114,6	8975732,6

Channel	Net Sales	Realized Net Sales	Mix	Return Rate
online	6073369,9	4988320,7	55,6%	17,9%
wholesale	1889540	1809690	20,2%	4,2%
retail	2007484,8	1734595,1	19,3%	13,4%
marketplace	515719,9	443126,8	4,9%	14,4%

For the CEO, the first things to see are current `net_sales` versus `realized_net_sales`, followed by the channel-performance table. The key takeaway is:

Online is the company's primary sales channel, generating 55.6% of realized net sales, but its 17.9% return rate is materially higher than every other channel. Wholesale is much smaller by volume, but its 4.2% return rate means that its realized sales are broadly comparable with Retail.

For the Head of Wholesale, the focus would be on how Wholesale compares with the other channels, particularly the evolution of `realized_net_sales` and `return_rate` over time. The key takeaway is:

Wholesale has substantially lower sales volume than Online, but its much lower return rate improves its realized performance, allowing it to surpass Retail in realized net sales.

2. Net sales and late-arriving returns

For this question, I created two queries in **business_questions/report.sql**.

2.1 What is wrong with the current model?

The main issue with the current model is that `net_sales` reflects the value of the sale **at the time the sale occurred**, but returns can arrive 30–90 days later and update `quantity_returned` on the original row.

For example, suppose a sale in January has:

- `net_sales = €90`
- `quantity_sold = 3`
- `quantity_returned = 0`

If two units are returned in March, `quantity_returned` will be updated to 2 on the original January row. However, the original `net_sales` value will remain €90.

This means that a dashboard based only on the current `net_sales` field cannot answer **"What are the realized net sales of historical transactions, given everything we know today?"** It effectively preserves the value calculated at the time of sale and does not reflect returns that arrive afterwards.

The case study explicitly highlights this problem: a dashboard can look correct when first built and then become misleading as new returns are applied to historical sales.

2.2 The solution

Instead of building the dashboard exclusively on the existing `net_sales` field, I introduced a second metric: **net sales as-of report date**, which is equivalent to the `realized_net_sales` metric introduced in Question 1.

This gives us two complementary metrics:

Net sales as-of sale date — `net_sales_at_sale`

This represents the net sales generated by the transaction at the time of sale, before any future returns are known.

```
net_sales_at_sale = gross_sale - taxes
```

This metric is **stable** and does not change when returns arrive. It is therefore useful when we want to preserve the original value of sales and avoid retroactively changing historical periods.

Net sales as-of report date — `realized_net_sales`

This represents the value of historical sales that remains after accounting for returns known at the time of the report.

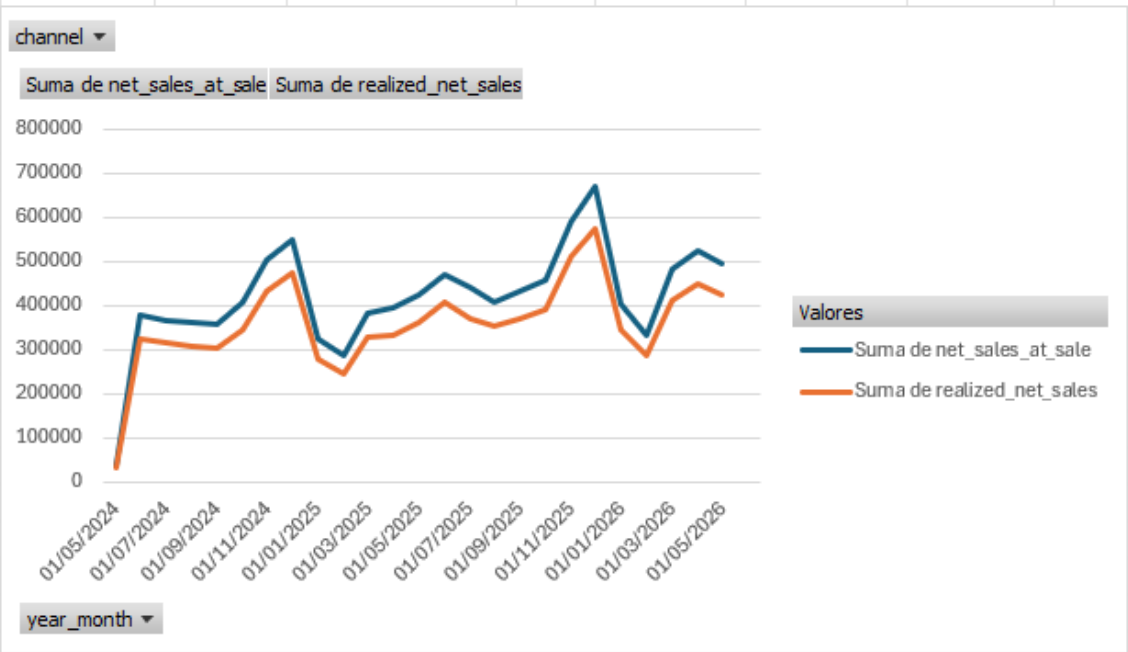
```
realized_net_sales = net_sales - value_of_returns_known_as_of_report_date
```

This metric changes over time as additional returns are recorded against historical sales.

An important caveat is that the current source model **cannot reconstruct what realized net sales looked like at an earlier reporting date**. The dataset only stores the current value of `quantity_returned` on the original sale row; it does not contain a return date or historical versions of `quantity_returned`. (This problem will be tackled in section **2.3 Another solution**)

Therefore, if a January sale has two returns by March, we can calculate its realized net sales as of March, but we cannot determine from the current source whether those returns had already occurred in February.

Using the output of query **2A**, I created a line chart showing both metrics over time, with the ability to filter by channel.



The two metrics answer different questions:

net_sales_at_sale — What sales value was generated when the transactions occurred?

realized_net_sales — Given everything we know as of the reporting date, how much of those historical sales remains after returns?

Immediately after a sale occurs, the two metrics will generally be equal because no future returns are known yet. As returns arrive, `realized_net_sales` can decrease for the affected historical months, while `net_sales_at_sale` remains unchanged.

This means the chart remains useful over time: the original sales trend stays stable, while the realized-sales trend progressively incorporates the returns that have become known.

2.3 Another solution — Model sales and returns as separate events

A more robust solution would be to separate sales events from return events, allowing us to preserve the history of when returns occurred.

I would model this using two fact tables:

`fct_sale_order_line`

Column	Description
sale_line_id	Unique identifier for the sale line
created_at	Timestamp of the original sale
channel	Sales channel
sku	Product SKU
shipment_id	Associated shipment
quantity_sold	Units originally sold
gross_sale	Gross sales value
taxes	Sales taxes
net_sales	Net sales at the time of sale

One row per original sale line. The sale itself should remain **immutable**, with no mutable `quantity_returned` field.

`fct_return`

One row per return event, or per returned sale line if multiple units are returned together.

Column	Description
return_id	Unique identifier for the return event
sale_line_id	Reference to the original sale line
return_date	Date/time the return occurred
quantity_returned	Number of units returned

This model preserves the **event history** instead of overwriting the original sale record.

It would then be possible to calculate realized net sales **as of any historical reporting date** by including only returns where return_date <= report_date (query **2B, it is a proposed model, doesn't run!**).

This solves the main limitation of the current source model and allows historical reports to remain reproducible. A report run today and a report run three months from now can both accurately represent what was known at their respective reporting dates.

3. Contribution margin

For this question, the queries can be found in **business_questions/contribution_margin.sql**.

3.1 Definition of contribution margin

Contribution margin (CM) measures how much revenue remains after deducting the variable/direct costs associated with making and fulfilling a sale.

I start with `realized_net_sales`, which represents revenue after taxes and the estimated value of returned units. From this, I subtract the product COGS, outbound shipping cost, and an estimated return cost:

```
contribution_margin = realized_net_sales - product_cogs - shipping_cost
- estimated_return_cost
```

Where:

```
product_cogs = units_retained_by_customer × dim_product.cost
```

Since the dataset does not provide a separate return-cost field, I estimate the return cost using the original `shipping_cost` when at least one unit is returned.

With this approach, contribution margin can be negative, particularly when all units on a sale line are returned. This is expected: the returned revenue is removed from `realized_net_sales`, while the original outbound shipping cost and estimated return shipping cost are still incurred.

3.2 Assumptions

I made the following assumptions:

1. **Realized net sales** deduct the estimated value of returned units from source `net_sales`.
2. **Product COGS** is calculated only for units retained by the customer. Returned units therefore do not contribute to final product COGS.
3. **Missing products:** Sale lines with an SKU that does not exist in `dim_product` are excluded because their product cost and category cannot be determined reliably.
4. **Missing shipments:** Sale lines with a `shipment_id` that does not exist in `fct_shipment` are excluded because their shipping cost cannot be determined reliably.

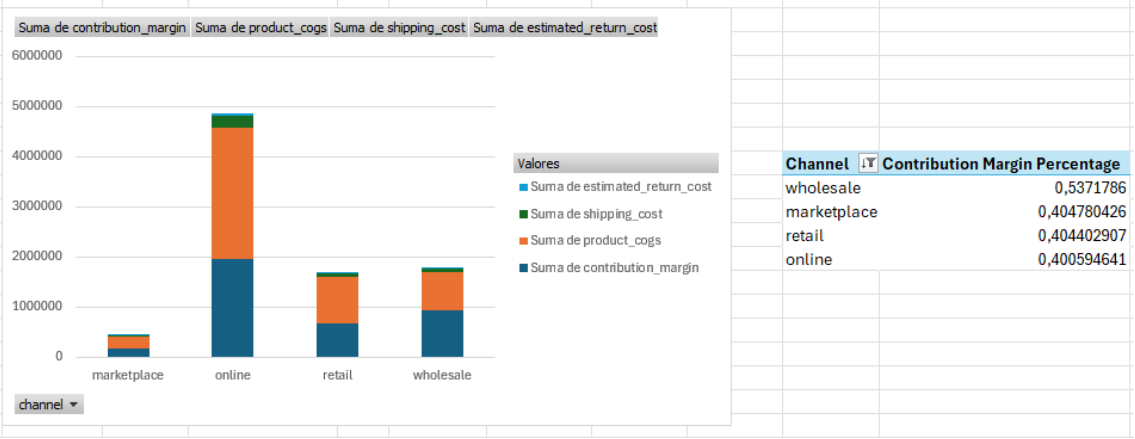
5. **Return cost:** When a return occurs, the original outbound `shipping_cost` is used as a proxy for the return cost because the source does not provide a separate return-cost field.
6. **Contribution margin:** CM is calculated as realized net sales minus product COGS, outbound shipping cost, and estimated return cost.

3.3 Dashboard view

Contribution margin by channel

With Query 3, I calculate contribution margin at the sale-line level.

Query **3A** aggregates the results by channel. I then extracted the results and created a stacked bar chart showing the contribution margin breakdown.



Channel	Contribution Margin Percentage
wholesale	0,5371786
marketplace	0,404780426
retail	0,404402907
online	0,400594641

The full bar represents **realized net sales**, with the following components showing how that revenue is allocated:

- Product COGS
- Shipping cost
- Estimated return cost
- Contribution margin

This provides a more meaningful view of channel profitability than revenue alone.

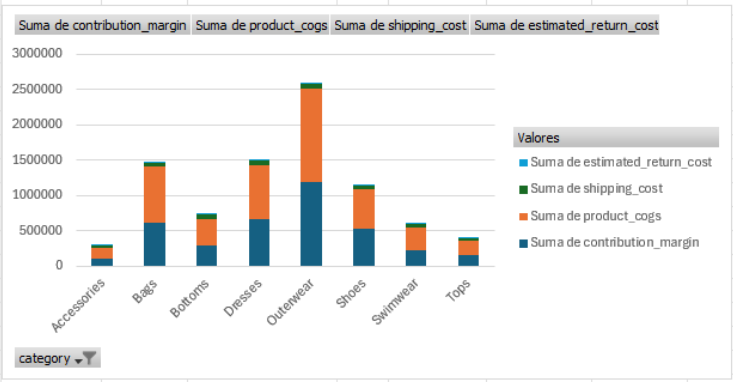
For example, **Online** looks very healthy when looking only at revenue because it is the company's largest channel. However, in the contribution margin view, approximately **60% of its realized net sales are consumed by costs**, primarily product COGS.

The difference is also clear when comparing **Retail and Wholesale**. Their revenue levels are relatively similar, but Wholesale has a significantly higher **contribution**

margin percentage (53% vs. 40% for Retail). This indicates that Wholesale retains substantially more value from each euro of realized net sales after direct costs.

Contribution margin by category

With Query **3B**, I aggregated contribution margin by product category and created a corresponding stacked bar chart.



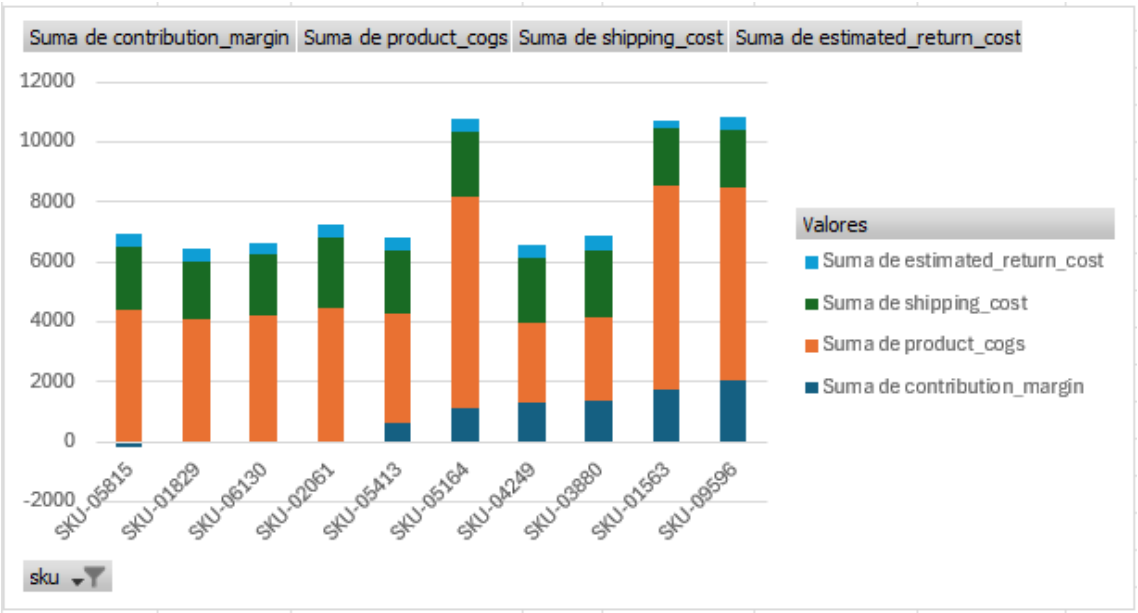
Category	Contribution Margin Percentage
Outerwear	0,46169298
Shoes	0,459049474
Dresses	0,435193371
Bags	0,41786964
Bottoms	0,399662322
Swimwear	0,375647381
Tops	0,369870733
Accessories	0,317833395

Outerwear has the highest contribution margin and contribution margin percentage, making it a strong-performing category from both a revenue and profitability perspective.

At the other end, **Accessories** has both the lowest revenue and the lowest contribution margin percentage. This makes it a category worth investigating further. Possible areas to examine would include pricing, product costs, return rates, and shipping costs before deciding whether a price change would be appropriate.

Contribution margin by SKU — lowest performers

With Query **3C**, I aggregated contribution margin by SKU and created a stacked bar chart showing the **10 SKUs with the lowest contribution margin**.



The three lowest-performing SKUs have **negative contribution margins**, meaning that, under the assumptions of this analysis, the company loses money on those sales after accounting for direct costs.

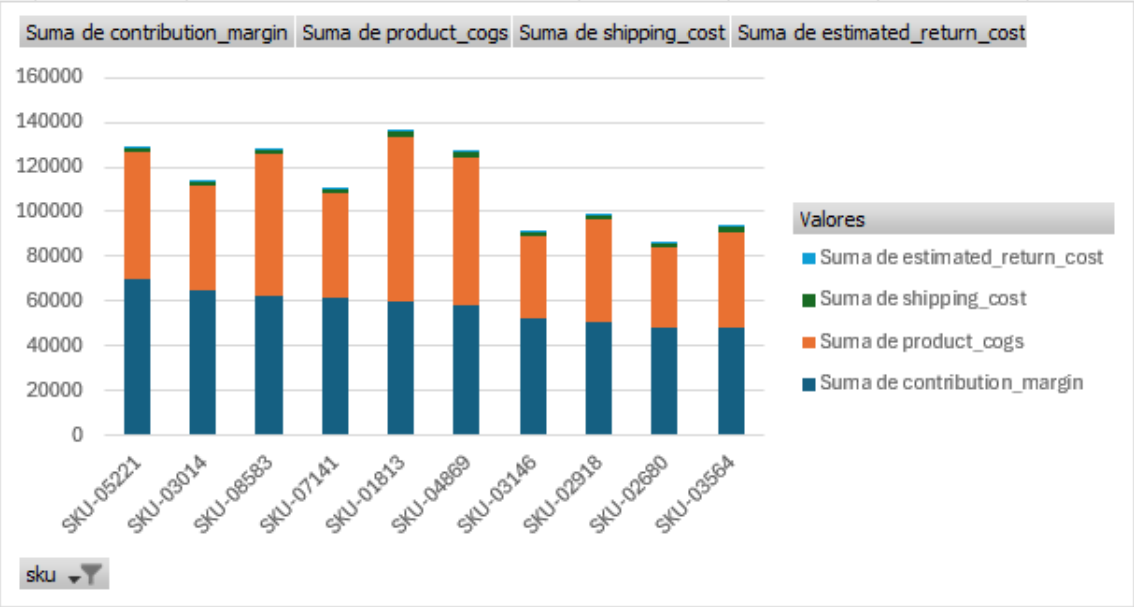
All three SKUs belong to the **Accessories** category, which is consistent with the category-level results from Query 3B.

The main drivers of the negative contribution margin appear to be **high product COGS combined with relatively low selling prices**, resulting in low realized net sales relative to the direct costs associated with fulfilling the orders.

These products would be good candidates for further investigation, particularly around product cost, pricing, returns, and fulfilment costs.

Contribution margin by SKU — highest performers

I also created a stacked bar chart showing the **10 SKUs with the highest contribution margin**.



These represent the products generating the most contribution margin under the assumptions of this analysis and can help identify products that are particularly valuable from a profitability perspective.

Together, the SKU-level views complement the channel and category analysis by showing **which individual products are driving or destroying contribution margin**, rather than looking only at aggregate performance.